

Excepții și fluxuri

Excepții în Java (Exceptions)

Excepțiile sunt mecanisme prin care Java gestionează erorile apărute în timpul execuției programului (runtime errors), fără a opri brusc aplicația. Ele permit:

- separarea logicii normale de tratarea erorilor
- control asupra fluxului de execuție în caz de probleme

Ierarhia excepțiilor

Toate excepțiile derivă din **Throwable**.

Structura principală:

Throwable

└─ Error (erori grave, nerecuperabile)

└─ Exception

 └─ Checked Exceptions

 └─ RuntimeException (Unchecked)

Tipuri de excepții

Checked Exceptions

- Verificate la compilare
- Trebuie tratate sau declarate (throws)

Exemple:

- IOException
- SQLException

```
void readFile() throws IOException {  
    // cod care poate arunca excepție  
}
```

Unchecked Exceptions (Runtime)

- Apar la execuție
- Nu sunt verificate de compilator

Exemple:

- NullPointerException
- ArrayIndexOutOfBoundsException
- ArithmeticException

```
int x = 10 / 0; // ArithmeticException
```

Errors

- Probleme grave (memorie, JVM)
- Nu se tratează

Exemple:

- OutOfMemoryError

Tratarea excepțiilor

try-catch

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Eroare: " + e.getMessage());  
}
```

Multiple catch

```
try {  
    // cod  
} catch (IOException e) {  
} catch (Exception e) {  
}
```

Multi-catch (Java 7+)

```
catch (IOException | SQLException e) {  
    System.out.println("Eroare IO sau DB");  
}
```

finally

Se execută indiferent dacă apare sau nu excepție.

```
try {  
    // cod  
} finally {  
    System.out.println("Cleanup");  
}
```

throw vs throws

throw (aruncă efectiv o excepție)

```
if (age < 18) {  
    throw new IllegalArgumentException("Minor");  
}
```

throws (declară)

```
void method() throws IOException {}
```

Excepții personalizate

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

//Utilizare:

if (age < 18) {
    throw new InvalidAgeException("Too young");
}
```

Try-with-resources (Java 7+)

Folosit pentru resurse (fișiere, streamuri)

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
}
```

Resursa se închide automat.

Fluxuri de intrare/ieșire (I/O Streams)

Fluxurile (streams) reprezintă un mod de a citi și scrie date între:

- program și fișiere
- program și rețea
- program și memorie

Tipuri principale de fluxuri

Byte Streams

- lucrează cu bytes (date brute)
- clase:
 - InputStream
 - OutputStream

Exemple:

- imagini
- fișiere binare

Character Streams

- lucrează cu caractere (text)
- clase:
 - Reader
 - Writer

Exemple:

- fișiere text

Exemple

Citire fișier (character stream)

```
import java.io.*;

BufferedReader br = new BufferedReader(new FileReader("test.txt"));
String line = br.readLine();
br.close();
```

Scriere fișier

```
BufferedWriter bw = new BufferedWriter(new FileWriter("test.txt"));
bw.write("Hello");
bw.close();
```

Fluxuri bufferizate

cresc performanța

- BufferedReader
- BufferedWriter
- BufferedInputStream

Fluxuri de nivel înalt

Data Streams

```
DataOutputStream dos = new DataOutputStream(new  
FileOutputStream("file.dat"));  
dos.writeInt(100);
```

Object Streams (Serializare)

```
ObjectOutputStream oos = new ObjectOutputStream(new  
FileOutputStream("obj.dat"));  
oos.writeObject(obj);
```

necesită Serializable

NIO (New I/O)

Introduse pentru performanță mai bună.

Clase importante:

- Path
- Files
- Channels
- Buffers

Exemplu NIO

```
import java.nio.file.*;  
  
String content = Files.readString(Path.of("file.txt"));
```

Diferență IO vs NIO

Caracteristică	IO	NIO
Model	stream	buffer
Blocking	da	non-blocking
Performanță	mai mică	mai mare

Exerciții

1. Creează o clasă `FileProcessor` care:
 - a. Citește un fișier text (`input.txt`)
 - b. Afișează numărul total de linii
 - c. Afișează doar liniile care conțin un cuvânt dat
 - d. Folosește:
 - `BufferedReader`
 - `try-with-resources`
 - e. Tratează:
 - `IOException`
 - f. Nu lăsa programul să crape
 - g. dacă fișierul nu există → afișează mesaj custom
2. Creează clasa `InvalidDataException`
Creează o clasă `UserService` care:
 - primește nume și vârstă

- validează:
 - nume $\neq$ null
 - vârstă > 0

Dacă datele sunt invalide → aruncă `InvalidDataException`

- dacă datele sunt valide → scrie într-un fișier `users.txt`
- format:

John - 25

- folosește `BufferedWriter`
- adaugă mai mulți utilizatori (append)

3. Creează o clasă `LogAnalyzer` care:

- Citește un fișier `log.txt` folosind: `Files.lines(Path.of(...))`
- filtrează liniile care conțin "ERROR"
- transformă în uppercase
- colectează într-o listă
- Afișează rezultatul.
- numără câte erori există

4. Creează o clasă `Serializable` numită `Product`, câmpuri `name` și `price`.

- Scrie o listă de produse într-un fișier (`products.dat`)
- Citește-le înapoi
- folosește:
 - `ObjectOutputStream`
 - `ObjectInputStream`

- tratează excepții
- validează ca $price > 0$, altfel aruncă excepție

5.

Ai un fișier numbers.txt cu valori:

10
20
abc
30
-5

- Citește fișierul
- Pentru fiecare linie:
 - încearcă să convertești la int
 - dacă nu → ignoră (dar loghează)
- păstrează doar numere pozitive
- dublează valorile
- calculează suma
- folosește:
 - try-with-resources
 - exception handling
- creează o metodă separată pentru parsare:
 - `Optional<Integer> parse(String line)`